

LỘ TRÌNH PHÁT TRIỂN CHI TIẾT: PROJECT TIMELINE

High School Learning Analytics Dashboard



Tổng quan dự án

- **Tổng số Giai đoạn:** 13 Giai đoạn
- **Thời gian:** 87 Ngày làm việc
- **Quy mô:** 45 Use Cases
- **Cơ sở dữ liệu:** 28 Bảng (Tables)



Giai đoạn 0: Khởi tạo Dự án

- **Thời gian:** Ngày 1–2 (2 ngày)
- **Use Cases:** Không có
- **Phụ thuộc:** Không có
- **Tóm tắt:** Chuẩn bị nền móng kỹ thuật: monorepo, Docker, cấu hình dev environment. Không viết logic nghiệp vụ.

1. Tạo cấu trúc Monorepo (SETUP)

Công việc chi tiết:

- Tạo thư mục gốc webquiz/ với client/, server/, prisma/, docker/
- Tạo .gitignore (node_modules, .env, dist, .prisma)
- Tạo .env.example với tất cả biến môi trường cần thiết
- Tạo README.md mô tả cách setup project

Cấu trúc thư mục:

webquiz/

```
|— client/      # React 18 + Vite + TypeScript
|— server/      # Node.js 20 + Express 4 + TypeScript
|— prisma/      # Prisma schema + migrations + seed
|— docker/      # Dockerfiles cho từng service
|— docs/        # 5 file HTML thiết kế
|— docker-compose.yml
|— .env.example
|— .gitignore
|— README.md
```

2. Khởi tạo Backend (server/) (BACKEND)

Công việc chi tiết:

- npm init -y trong thư mục server/
- Cài dependencies: express, typescript, ts-node-dev, dotenv, cors, helmet, morgan, compression
- Cài dev-deps: @types/express, @types/node, @types/cors, eslint, prettier
- Tạo tsconfig.json (strict mode, paths alias)
- Tạo .eslintrc.js + .prettierrc
- Tạo cấu trúc thư mục src/ theo module pattern
- Viết server.ts — Express app Hello World + health check endpoint
- Thêm script dev: ts-node-dev --respawn src/server.ts

Cấu trúc thư mục:

```
server/src/
├── config/      # database.ts, redis.ts, minio.ts, env.ts
├── middlewares/ # auth.ts, errorHandler.ts, rateLimiter.ts, validate.ts, activityLogger.ts
├── modules/
│   ├── auth/    # controller, service, routes, validation
│   ├── user/
│   ├── exam/
│   ├── question/
│   ├── analytics/
│   ├── ai/
│   ├── notification/
│   ├── class/
│   ├── academic/
│   └── system/
├── utils/      # logger.ts (Winston), helpers.ts, constants.ts
├── types/      # express.d.ts, common.ts
├── app.ts      # Express app setup (middlewares, routes)
└── server.ts   # Entry point (listen port)
```

3. Khởi tạo Frontend (client/) (FRONTEND)

Công việc chi tiết:

- npm create vite@latest . -- --template react-ts trong client/
- Cài Tailwind CSS: npm i -D tailwindcss @tailwindcss/vite + cấu hình
- Cài core deps: react-router-dom, axios, @tanstack/react-query, zustand
- Cài UI deps: react-hook-form, zod, @hookform/resolvers, react-hot-toast
- Cài chart deps: chart.js, react-chartjs-2, d3 (cho Knowledge Graph)
- Cài realtime: socket.io-client

- Tạo cấu trúc thư mục src/ theo feature pattern
- Viết App.tsx cơ bản với React Router (placeholder pages)
- Cấu hình axios instance với baseURL + interceptors skeleton

Cấu trúc thư mục:

```

client/src/
├── assets/      # images, fonts
├── components/  # Button, Input, Modal, Table, Badge, Card, Spinner...
├──   ├── ui/    # Primitive UI components
├──   └── shared/ # QuestionCard, SubjectSelect, NotificationBell...
├── layouts/     # AuthLayout, StudentLayout, TeacherLayout, AdminLayout, ParentLayout
├── pages/
├──   ├── auth/  # LoginPage, ForgotPasswordPage, ResetPasswordPage
├──   └── student/ # DashboardPage, ExamListPage, TakeExamPage, ResultPage,
├── AIPracticePage
├──   ├── teacher/ # DashboardPage, QuestionBankPage, CreateExamPage, ClassesPage
├──   ├── parent/  # DashboardPage, ChildResultsPage
├──   ├── admin/   # DashboardPage, UsersPage, AcademicPage, SystemPage
├──   └── common/  # NotFoundPage, ForbiddenPage, ProfilePage
├── hooks/        # useAuth, useExam, useDebounce, useSocket...
├── services/     # api.ts (axios), auth.api.ts, exam.api.ts, user.api.ts...
├── stores/       # authStore.ts (zustand), uiStore.ts
├── types/        # user.ts, exam.ts, question.ts, api.ts
├── utils/        # format.ts, validate.ts, constants.ts
├── App.tsx
└── main.tsx

```

4. Docker Compose cho Development (DEVOPS)

Công việc chi tiết:

- Viết docker-compose.yml với 3 services: postgres, redis, minio
- PostgreSQL 16: port 5432, volume cho data persistence, init script tạo database
- Redis 7: port 6379, password, volume
- MinIO: port 9000 (API) + 9001 (Console), access/secret key, volume
- Tạo .env từ .env.example với credentials local
- Test: docker compose up -d → 3 services running
- Verify: kết nối pgAdmin/psql, redis-cli ping, MinIO console

Code Example:

```

# docker-compose.yml
services:

```

```
postgres:
  image: postgres:16-alpine
  ports: ["5432:5432"]
  environment:
    POSTGRES_DB: webquiz
    POSTGRES_USER: webquiz
    POSTGRES_PASSWORD: ${DB_PASSWORD}
  volumes: [pgdata:/var/lib/postgresql/data]

redis:
  image: redis:7-alpine
  ports: ["6379:6379"]
  command: redis-server --requirepass ${REDIS_PASSWORD}

minio:
  image: minio/minio:latest
  ports: ["9000:9000", "9001:9001"]
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: ${MINIO_ACCESS_KEY}
    MINIO_ROOT_PASSWORD: ${MINIO_SECRET_KEY}
  volumes: [minio_data:/data]

volumes:
  pgdata:
  minio_data:
```

5. Git + GitHub Setup (DEVOPS)

Công việc chi tiết:

- git init + initial commit
- Tạo GitHub repository (private)
- Branch strategy: main (production) → develop (staging) → feature/* (dev)
- Tạo .github/workflows/ci.yml skeleton (chỉ echo, chưa cần pipeline thật)
- Push initial commit lên GitHub

 **Deliverable** — **Giai đoạn 0:** docker compose up → 3 services chạy. npm run dev ở cả client + server → Hello World. Git repo on GitHub.



Giai đoạn 1: Database + Prisma Schema

- **Thời gian:** Ngày 3–5 (3 ngày)
- **Use Cases:** Không có

- **Phụ thuộc:** GD 0
- **Tóm tắt:** Toàn bộ 28 bảng ERD → Prisma schema, migrate vào PostgreSQL, seed data, kết nối Redis + MinIO.

1. Viết Prisma Schema — 6 Enum Types (BACKEND)

Công việc chi tiết:

- Cài Prisma: `npm i prisma @prisma/client + npx prisma init`
- Cấu hình datasource postgresql trong `schema.prisma`
- Định nghĩa enum `UserStatus { ACTIVE INACTIVE SUSPENDED }`
- Định nghĩa enum `QuestionKind { SINGLE_CHOICE MULTIPLE_CHOICE }`
- Định nghĩa enum `ExamStatus { DRAFT PUBLISHED SCHEDULED CLOSED }`
- Định nghĩa enum `ScheduleStatus { PENDING ACTIVE COMPLETED CANCELLED }`
- Định nghĩa enum `AttemptStatus { IN_PROGRESS SUBMITTED GRADED }`
- Định nghĩa enum `BackupStatus { IN_PROGRESS COMPLETED FAILED }`

2. Viết Prisma Schema — 28 Models (BACKEND)

Công việc chi tiết:

- User group (4 models): `Role`, `User` (@unique email, username), `ParentStudent`, `PasswordResetToken` (@unique token)
- Academic group (4 models): `AcademicYear`, `Semester`, `Class`, `ClassStudent` (composite PK)
- Curriculum group (4 models): `Subject` (@unique code), `Chapter`, `Topic`, `TopicRelation`
- Question group (3 models): `Question`, `QuestionOption`, `QuestionTag`
- Exam group (4 models): `Exam`, `ExamQuestion` (composite PK), `ExamSchedule`, `ExamAssignment`
- Result group (2 models): `ExamAttempt` (@@index [studentId, examId]), `AttemptAnswer`
- AI group (2 models): `AiPracticeSession`, `AiGeneratedQuestion`
- System group (5 models): `Notification` (@@index [userId, isRead]), `ActivityLog`, `SystemConfig` (@unique configKey), `Backup`, `WebPushSubscription`
- Thiết lập tất cả relations: @relation, onDelete cascade/setNull theo ERD

3. Migration + Verify (BACKEND)

Công việc chi tiết:

- Chạy `npx prisma migrate dev --name init` — tạo 28 bảng
- Verify trong pgAdmin: kiểm tra 28 bảng, FK constraints, indexes, enum types
- Chạy `npx prisma generate` — tạo Prisma Client
- Test import Prisma Client trong server code — query đơn giản

4. Seed Data (BACKEND)

Công việc chi tiết:

- Tạo `prisma/seed.ts`
- Seed 4 roles: student, parent, teacher, admin

- Seed 1 admin account: admin@school.edu.vn / Admin@123 (bcrypt hash)
- Seed 5 môn học: Toán, Vật lý, Hóa học, Sinh học, Tiếng Anh (với code: MATH, PHY, CHEM, BIO, ENG)
- Seed 1 năm học 2025-2026 (is_current: true) + 2 học kỳ
- Seed 2-3 chương cho mỗi môn + 3-5 chủ đề mỗi chương
- Seed vài topic_relations mẫu cho Knowledge Graph
- Cấu hình seed script trong package.json: "prisma": {"seed": "ts-node prisma/seed.ts"}
- Chạy npx prisma db seed — verify data trong pgAdmin

5. Kết nối Redis + MinIO (BACKEND)

Công việc chi tiết:

- Cài ioredis: npm i ioredis
- Tạo config/redis.ts: kết nối Redis với password, error handling, reconnect strategy
- Test Redis: set/get key, PING → PONG
- Cài MinIO client: npm i minio
- Tạo config/minio.ts: kết nối MinIO, tạo bucket "uploads" nếu chưa có
- Test MinIO: upload 1 file text → download → verify nội dung
- Tạo config/database.ts: export Prisma Client singleton

 **Deliverable** — **Giai đoạn 1:** 28 bảng trong PostgreSQL + seed data. Prisma Client generate OK. Redis PING → PONG. MinIO upload/download OK.

Giai đoạn 2: Auth Service

- **Thời gian:** Ngày 6-10 (5 ngày)
- **Use Cases:** UC01, UC02, UC03, UC04
- **Phụ thuộc:** GĐ 1
- **Tóm tắt:** Xác thực JWT cho 4 vai trò, refresh token qua Redis, đặt lại mật khẩu, protected routes.

1. Backend — Auth Module (BACKEND)

API Endpoints:

- **[POST]** /api/auth/login - Đăng nhập → Access Token (15m) + Refresh Token (7d, Redis)
- **[POST]** /api/auth/logout - Xóa Refresh Token khỏi Redis
- **[POST]** /api/auth/refresh - Refresh Token → Access Token mới
- **[POST]** /api/auth/forgot-password - Gửi email reset (tạo token trong DB, TTL 15m)
- **[POST]** /api/auth/reset-password - Verify token → đổi password → xóa token

Công việc chi tiết:

- Tạo modules/auth/auth.validation.ts: Zod schemas cho login, register, refreshToken, forgotPassword, resetPassword
- Tạo modules/auth/auth.service.ts: login (verify bcrypt → issue JWT), logout (delete Redis key), refreshToken, forgotPassword, resetPassword

- Tạo modules/auth/auth.controller.ts: request handlers gọi service
- Tạo modules/auth/auth.routes.ts: mount routes

2. Backend — Middlewares (BACKEND)

Công việc chi tiết:

- middlewares/auth.ts: verify JWT từ Authorization header → gán req.user = {id, role, email}
- middlewares/authorize.ts: roleMiddleware("admin","teacher") → check req.user.role
- middlewares/errorHandler.ts: global error handler → format {success:false, message, errors}
- middlewares/validate.ts: nhận Zod schema → validate req.body → throw nếu invalid
- middlewares/rateLimiter.ts: rate-limit 5 request/15 phút cho login (dùng express-rate-limit + Redis store)
- middlewares/activityLogger.ts: ghi action vào activity_logs (user, action, entity, ip)

3. Backend — User Profile (BACKEND)

API Endpoints:

- **[GET]** /api/users/me - Lấy thông tin user hiện tại
- **[PUT]** /api/users/me - Cập nhật full_name, phone
- **[PUT]** /api/users/me/password - Đổi mật khẩu (verify old → hash new)
- **[POST]** /api/users/me/avatar - Upload ảnh đại diện → MinIO

Công việc chi tiết:

- Tạo modules/user/user.service.ts: getProfile, updateProfile, changePassword, uploadAvatar
- Upload avatar: nhận file → validate (jpg/png, max 2MB) → upload MinIO → lưu URL vào users.avatar

4. Frontend — Auth Pages (FRONTEND)

Công việc chi tiết:

- Tạo pages/auth/LoginPage.tsx: form email + password (react-hook-form + zod validation)
- UI: split layout — left: gradient brand panel, right: form (theo mockup ui-ux-design.html)
- Submit → gọi API login → lưu tokens vào localStorage → redirect theo role
- Hiển thị lỗi: "Sai email hoặc mật khẩu", "Tài khoản bị khóa"
- Tạo pages/auth/ForgotPasswordPage.tsx: form nhập email → gọi API → hiện thông báo "Kiểm tra email"
- Tạo pages/auth/ResetPasswordPage.tsx: form nhập mật khẩu mới (từ URL chứa token)

5. Frontend — Auth System (FRONTEND)

Công việc chi tiết:

- Tạo stores/authStore.ts (Zustand): user, tokens, login(), logout(), isAuthenticated
- Tạo services/api.ts: axios instance với baseURL, interceptors (gắn Bearer token, auto refresh khi 401)

- Tạo hooks/useAuth.ts: wrap authStore, thêm logic check token expiry
- Tạo components/ProtectedRoute.tsx: check isAuthenticated + role → redirect /login hoặc /403
- Tạo layouts/AuthLayout.tsx: layout cho login/reset (no sidebar)
- Tạo layouts/DashboardLayout.tsx: sidebar + topbar + content area (responsive)
- Cấu hình React Router: public routes (/login, /forgot-password) + protected routes (/:role/*)

 **Deliverable — Giai đoạn 2:** Đăng nhập 4 vai trò, JWT + refresh token, đặt lại mật khẩu. Frontend: Login page, protected routes, dashboard layout skeleton.

Giai đoạn 3: Admin Portal

- **Thời gian:** Ngày 11–17 (7 ngày)
- **Use Cases:** UC37, UC38, UC39, UC40, UC41, UC42, UC43, UC44, UC45
- **Phụ thuộc:** GĐ 2
- **Tóm tắt:** Admin quản lý users, môn học, năm học, cấu hình hệ thống, giám sát, backup. Tạo dữ liệu nền cho các module sau.

1. Backend — User Management (UC37–39) (BACKEND)

API Endpoints:

- **[GET]** /api/admin/users - Danh sách users (search, filter role/status, pagination)
- **[POST]** /api/admin/users - Tạo user mới (hash password, gán role)
- **[PUT]** /api/admin/users/:id - Cập nhật user (name, email, phone, status)
- **[DELETE]** /api/admin/users/:id - Soft delete (đổi status → SUSPENDED)
- **[POST]** /api/admin/users/import - Import Excel (parse xlsx → validate → bulk create)
- **[GET]** /api/admin/users/import-template - Download file Excel mẫu
- **[PUT]** /api/admin/users/:id/role - Thay đổi vai trò

Công việc chi tiết:

- Service: CRUD users với Prisma, bcrypt hash cho password mới
- Import Excel: dùng thư viện xlsx — parse file → validate từng row (email unique, role valid) → trả danh sách lỗi chi tiết
- Validation: email format, username 3–50 chars, password 8+ chars (upper+lower+number)
- Pagination: page, limit, sort, search (full-text trên name + email)
- Activity log: mọi CUD action được ghi tự động

2. Backend — Academic Management (UC40–41) (BACKEND)

API Endpoints:

- **[GET]** /api/admin/subjects - Danh sách môn học
- **[POST]** /api/admin/subjects - Tạo môn học (name, code unique)
- **[PUT]** /api/admin/subjects/:id - Cập nhật môn học
- **[GET]** /api/admin/academic-years - Danh sách năm học
- **[POST]** /api/admin/academic-years - Tạo năm học + tự động tạo 2 học kỳ

- **[PUT]** /api/admin/academic-years/:id - Cập nhật (set is_current)
- **[GET]** /api/admin/semesters - Danh sách học kỳ

3. Backend — System Management (UC42–45) (BACKEND)

API Endpoints:

- **[GET]** /api/admin/configs - Lấy tất cả system_configs
- **[PUT]** /api/admin/configs - Cập nhật batch configs
- **[GET]** /api/admin/monitoring - CPU, RAM, uptime, active connections
- **[GET]** /api/admin/activity-logs - Nhật ký (filter user, action, date range, pagination)
- **[POST]** /api/admin/backups - Tạo backup (pg_dump → gzip → MinIO)
- **[POST]** /api/admin/backups/:id/restore - Restore từ MinIO → pg_restore
- **[GET]** /api/admin/backups - Danh sách backups

Công việc chi tiết:

- Monitoring: dùng Node.js os module (cpus, totalmem, freemem, uptime) + process.memoryUsage()
- Backup: child_process.exec pg_dump → stream to gzip → upload MinIO → save record
- Restore: download từ MinIO → gunzip → pg_restore

4. Frontend — Admin Dashboard (FRONTEND)

Công việc chi tiết:

- Tạo pages/admin/DashboardPage.tsx: cards tổng quan (total users, exams, subjects, system health)
- Tạo pages/admin/UsersPage.tsx: bảng DataTable (search, filter dropdown, pagination, bulk actions)
- Modal tạo/sửa user (react-hook-form, zod validation)
- Import Excel: upload zone → preview table → confirm/cancel
- Tạo pages/admin/AcademicPage.tsx: tabs Môn học | Năm học | Học kỳ — mỗi tab có DataTable + CRUD modal
- Tạo pages/admin/SystemPage.tsx: tabs Cấu hình | Hiệu năng | Nhật ký | Sao lưu
- Biểu đồ real-time CPU/RAM (Chart.js line chart, polling 5s)
- Activity logs: bảng với filter + date range picker
- Backup: danh sách + nút "Sao lưu ngay" → loading → success toast

 **Deliverable — Giai đoạn 3:** Admin portal đầy đủ UC37–45. Dữ liệu nền: users, subjects, academic years sẵn sàng cho các module tiếp theo.

? Giai đoạn 4: Ngân hàng Câu hỏi

- **Thời gian:** Ngày 18–23 (6 ngày)
- **Use Cases:** UC23, UC24, UC25
- **Phụ thuộc:** GĐ 3
- **Tóm tắt:** Giáo viên tạo, import, gắn tag câu hỏi trắc nghiệm. Ngân hàng sẵn sàng để tạo

để kiểm tra.

1. Backend — Question Service (BACKEND)

API Endpoints:

- **[GET]** /api/questions - Danh sách câu hỏi (filter: subject, chapter, topic, difficulty, keyword)
- **[POST]** /api/questions - Tạo câu hỏi + 4 options + correct answer + explanation
- **[PUT]** /api/questions/:id - Cập nhật câu hỏi
- **[DELETE]** /api/questions/:id - Xóa câu hỏi (check không thuộc exam đang active)
- **[POST]** /api/questions/import - Import từ Excel (content, A, B, C, D, correct, explanation, difficulty)
- **[GET]** /api/questions/import-template - Download file mẫu Excel
- **[POST]** /api/questions/:id/tags - Thêm tags cho câu hỏi
- **[DELETE]** /api/questions/:id/tags/:tagId - Xóa tag

Công việc chi tiết:

- Tạo câu hỏi: validate content không rỗng, đúng 4 options, ít nhất 1 correct, difficulty 1–5
- Import Excel: parse xlsx → validate headers → validate từng row → báo lỗi chi tiết (row number + reason)
- Filter: cascading subject → chapter → topic, multi-difficulty, keyword search (PostgreSQL ILIKE)
- Pagination: cursor-based hoặc offset (default 20/page)

2. Backend — Curriculum Endpoints (BACKEND)

API Endpoints:

- **[GET]** /api/subjects - Danh sách môn học (public cho teacher/student)
- **[GET]** /api/subjects/:id/chapters - Chương theo môn
- **[GET]** /api/chapters/:id/topics - Chủ đề theo chương
- **[POST]** /api/chapters - Tạo chương (teacher)
- **[POST]** /api/topics - Tạo chủ đề (teacher)
- **[GET]** /api/topics/:id/relations - Các chủ đề liên quan (Knowledge Graph edges)

3. Frontend — Question Bank Page (FRONTEND)

Công việc chi tiết:

- Tạo pages/teacher/QuestionBankPage.tsx: layout 2 cột (filter sidebar + question list)
- Filter sidebar: cascading Select (Môn → Chương → Chủ đề), difficulty multi-select, keyword search
- Question list: cards hiển thị content (truncated), difficulty badge, tags, subject/chapter
- Click card → expand chi tiết: full content, 4 options (highlight correct), explanation
- Tạo components/shared/QuestionFormModal.tsx: form phức tạp cho tạo/sửa câu hỏi
- Form fields: rich text content, 4 option inputs, radio chọn correct, difficulty slider, subject/chapter/topic selects, tags input

- Import Excel: dropzone upload → loading → preview table (highlight errors red) → confirm button
- Tạo components/shared/SubjectChapterTopicSelect.tsx: cascading 3-level dropdown (tái sử dụng)
- Tạo components/shared/DifficultyBadge.tsx: badge 5 mức (xanh/xanh lá/vàng/cam/đỏ)

 **Deliverable — Giai đoạn 4:** Giáo viên CRUD câu hỏi, import Excel, gắn tag, filter/search. Ngân hàng câu hỏi sẵn sàng cho để kiểm tra.



Giai đoạn 5: Bài Kiểm tra + Lớp học

- **Thời gian:** Ngày 24–31 (8 ngày)
- **Use Cases:** UC26, UC27, UC28, UC29
- **Phụ thuộc:** GĐ 4
- **Tóm tắt:** Giáo viên tạo đề KT, đặt lịch auto mở/đóng, phân công cho lớp. Quản lý lớp học + danh sách HS.

1. Backend — Exam Service (BACKEND)

API Endpoints:

- **[POST]** /api/exams - Tạo bài KT (title, subject, duration, config)
- **[GET]** /api/exams - Danh sách bài KT của teacher (filter status)
- **[GET]** /api/exams/:id - Chi tiết bài KT + câu hỏi
- **[PUT]** /api/exams/:id - Cập nhật (chỉ khi status = DRAFT)
- **[POST]** /api/exams/:id/questions - Thêm câu hỏi vào đề (manual pick hoặc random)
- **[PUT]** /api/exams/:id/publish - Publish → đổi status DRAFT → PUBLISHED
- **[POST]** /api/exams/:id/schedule - Đặt lịch (start_time, end_time)
- **[POST]** /api/exams/:id/assign - Phân công cho 1+ lớp
- **[GET]** /api/exams/:id/assignments - Danh sách lớp đã phân công

Công việc chi tiết:

- Random câu hỏi: nhận {subjectId, chapterIds?, difficulty?, count} → random pick từ ngân hàng
- Shuffle logic: khi config.shuffle=true, lưu seed per-student để reproducible
- Cron job: mỗi phút check exam_schedules — nếu now >= start_time → status PUBLISHED, nếu now >= end_time → status CLOSED
- Validation: duration > 0, total_questions > 0, passing_score 0–10

2. Backend — Class Service (BACKEND)

API Endpoints:

- **[GET]** /api/classes - Danh sách lớp (teacher thấy lớp mình, admin thấy tất cả)
- **[POST]** /api/classes - Tạo lớp (name, grade_level, semester, subject)
- **[PUT]** /api/classes/:id - Cập nhật lớp
- **[GET]** /api/classes/:id/students - Danh sách HS trong lớp

- **[POST]** /api/classes/:id/students - Thêm HS vào lớp (userIds[])
- **[DELETE]** /api/classes/:id/students/:studentId - Xóa HS khỏi lớp
- **[POST]** /api/classes/:id/students/import - Import HS từ Excel

3. Frontend — Create Exam Wizard (FRONTEND)

Công việc chi tiết:

- Tạo pages/teacher/CreateExamPage.tsx: step wizard 5 bước
- Step 1 — Thông tin: title, subject select, duration (phút), passing score
- Step 2 — Chọn câu hỏi: browse ngân hàng (reuse QuestionBank filter) + tích chọn, HOẶC random (nhập criteria + count)
- Step 3 — Cấu hình: shuffle on/off, show result after submit on/off, max attempts
- Step 4 — Đặt lịch (optional): date/time picker cho start + end
- Step 5 — Phân công: multi-select classes → review summary → Create button
- Preview: xem để trước khi publish (read-only exam view)

4. Frontend — Classes & Exams Pages (FRONTEND)

Công việc chi tiết:

- Tạo pages/teacher/ClassesPage.tsx: danh sách lớp cards, click → chi tiết
- Chi tiết lớp: bảng HS + nút thêm/xóa + import Excel
- Tạo pages/teacher/ExamsPage.tsx: bảng bài KT với status badge (draft/published/scheduled/closed)
- Row actions: Edit (draft only), Schedule, Assign, View Results, Delete (draft only)

 **Deliverable — Giai đoạn 5:** Tạo đề KT step-by-step, đặt lịch auto, phân công lớp. Quản lý lớp học + HS. Sẵn sàng cho HS làm bài.



Giai đoạn 6: Làm bài Kiểm tra

- **Thời gian:** Ngày 32–39 (8 ngày)
- **Use Cases:** UC05, UC06, UC07, UC08, UC09
- **Phụ thuộc:** GĐ 5
- **Tóm tắt:** Học sinh làm bài KT online: đồng hồ đếm ngược, auto-save, auto-submit, xem kết quả chi tiết.

1. Backend — Exam Taking (BACKEND)

API Endpoints:

- **[GET]** /api/student/exams - DS bài KT được phân công (filter: upcoming, in_progress, completed)
- **[POST]** /api/student/exams/:id/start - Bắt đầu làm bài → tạo attempt, trả câu hỏi (KHÔNG có đáp án)
- **[PUT]** /api/student/attempts/:id/save - Auto-save câu trả lời vào Redis
- **[POST]** /api/student/attempts/:id/submit - Nộp bài → tính điểm → lưu DB → xóa Redis

draft

- **[GET]** /api/student/attempts/:id/result - Kết quả: score, per-question detail + explanation
- **[GET]** /api/student/attempts - Lịch sử tất cả bài đã làm

Công việc chi tiết:

- Start exam: check đã có attempt IN_PROGRESS chưa (resume nếu có), check exam status = PUBLISHED, check student thuộc assigned class
- Trả câu hỏi: shuffle nếu config, KHÔNG include is_correct / explanation
- Auto-save: lưu Redis key attempt:{id}:answers = JSON {questionId: selectedOptionId}, TTL = duration + 5 phút buffer
- Submit: lấy answers từ Redis (hoặc request body), match với correct answers → tính total_score, time_spent
- Tạo attempt_answers records: mỗi câu ghi selected_option_id, is_correct, time_spent_sec, answer_changes
- Auto-submit cron: mỗi phút check attempts WHERE status=IN_PROGRESS AND started_at + duration < now() → auto submit, is_auto_submitted = true
- Result: chỉ trả nếu exam.show_result = true HOẶC teacher đã graded
- Anti-cheat: max 1 active attempt per exam per student, validate time_spent <= duration

2. Frontend — Exam Taking (Màn hình quan trọng nhất) (FRONTEND)

Công việc chi tiết:

- Tạo pages/student/ExamListPage.tsx: cards bài KT grouped by status (sắp diễn ra / đang làm / đã nộp)
- Card hiển thị: tên, môn, duration, deadline, status badge
- Tạo pages/student/TakeExamPage.tsx — CRITICAL PAGE:
- Header: tên bài + đồng hồ đếm ngược (MM:SS), đổi đỏ khi < 5 phút, nhấp nháy khi < 1 phút
- Sidebar phải: bảng số câu (grid 5×N), click chuyển câu. Màu: xám=chưa, xanh=đã trả lời, vàng=đánh dấu
- Content giữa: câu hỏi (render HTML nếu rich text) + 4 radio options
- Footer: nút Previous / Next / Đánh dấu / Nộp bài
- Auto-save: mỗi 30 giây gọi PUT /save → hiện "Đã lưu" nhỏ
- Confirm dialog nộp bài: "Bạn còn X câu chưa trả lời. Xác nhận nộp?"
- Xử lý mất kết nối: hiện banner warning, queue saves, retry khi reconnect
- Xử lý hết giờ: auto-submit + redirect tới result page
- Tạo pages/student/ExamResultPage.tsx:
- Tổng quan: điểm (lớn, nổi bật), thời gian, đúng/sai/bỏ qua, đạt/không đạt
- Chi tiết: từng câu: nội dung, option đã chọn (highlight), đáp án đúng (highlight), giải thích
- Màu: xanh (đúng), đỏ (sai), xám (bỏ qua)

 **Deliverable — Giai đoạn 6:** Học sinh làm bài KT online hoàn chỉnh: đếm ngược, auto-save, auto-submit, kết quả chi tiết. Core feature hoàn thành.



Giai đoạn 7: Analytics & Dashboard

- **Thời gian:** Ngày 40–50 (11 ngày)
- **Use Cases:** UC10, UC11, UC12, UC13, UC14, UC15, UC30, UC31, UC32, UC33, UC34, UC35, UC36
- **Phụ thuộc:** GĐ 6
- **Tóm tắt:** Dashboard phân tích cho Student (điểm mạnh/yếu, Knowledge Graph) và Teacher (hiệu suất lớp, phân bố điểm, xuất báo cáo).

1. Backend — Student Analytics (UC10–15) (BACKEND)

API Endpoints:

- **[GET]** /api/student/dashboard - Tổng hợp: total exams, avg score, recent results, trend 30/60/90d
- **[GET]** /api/student/analytics/strengths - % đúng per topic → radar chart data
- **[GET]** /api/student/analytics/time - Avg time per question, per topic
- **[GET]** /api/student/analytics/patterns - Answer changes, skipped, time distribution
- **[GET]** /api/student/analytics/knowledge-graph - Nodes (topics + mastery) + edges (topic_relations)
- **[GET]** /api/student/attempts - Lịch sử bài KT (pagination, filter subject, date range)

Công việc chi tiết:

- Dashboard aggregation: Prisma groupBy + aggregate queries
- Strengths: GROUP BY topic_id, AVG(is_correct) → trả {topicName, accuracy, totalQuestions}
- Knowledge Graph: query topics + topic_relations + join với attempt results → node color = mastery level (red < 40%, yellow 40–70%, green > 70%)
- Redis cache: cache dashboard + strengths data, TTL 5 phút, invalidate khi submit mới

2. Backend — Teacher Analytics (UC30–36) (BACKEND)

API Endpoints:

- **[GET]** /api/teacher/classes/:id/dashboard - Class overview: enrolled, avg score, pass rate
- **[GET]** /api/teacher/classes/:id/analytics/performance - Trend: avg score per exam over time
- **[GET]** /api/teacher/exams/:id/analytics/distribution - Histogram bins [0-2, 2-4, 4-6, 6-8, 8-10]
- **[GET]** /api/teacher/classes/:id/analytics/weak-students - HS có avg < threshold liên tục 3 bài
- **[GET]** /api/teacher/analytics/compare-classes - So sánh avg score giữa nhiều lớp
- **[GET]** /api/teacher/exams/:id/results - Bảng kết quả từng HS (sortable)
- **[POST]** /api/teacher/reports/export - Xuất PDF/Excel → MinIO → trả download URL

Công việc chi tiết:

- Xuất PDF: dùng PDFKit — header trường, tiêu đề báo cáo, bảng kết quả, biểu đồ (chart as image)
- Xuất Excel: dùng xlsx — sheet "Tổng hợp" + sheet "Chi tiết"
- Upload file xuất vào MinIO bucket "reports" → trả signed URL (expire 1h)

3. Frontend — Student Dashboard (FRONTEND)

Công việc chi tiết:

- Tạo pages/student/DashboardPage.tsx: grid layout responsive
- Row 1: Stat cards — Tổng bài KT, Điểm TB, Bài gần nhất, Streak
- Row 2 left: Line chart xu hướng điểm (Chart.js, 30/60/90 ngày toggle)
- Row 2 right: Radar chart điểm mạnh/yếu (per topic, max 8 axes)
- Row 3 left: Bar chart time-on-task (avg seconds per topic)
- Row 3 right: Bảng lịch sử bài KT gần nhất (5 rows, link "Xem tất cả")
- Knowledge Graph page: D3.js force-directed graph (nodes = topics, edges = relations, node size = attempts count, node color = mastery)
- Click node → show detail panel: topic name, accuracy %, recent questions

4. Frontend — Teacher Dashboard (FRONTEND)

Công việc chi tiết:

- Tạo pages/teacher/DashboardPage.tsx: class selector dropdown ở top
- Stat cards: Sĩ số, Điểm TB, Tỷ lệ đạt, Bài KT gần nhất
- Line chart: hiệu suất lớp qua các bài KT (multiple series nếu compare classes)
- Histogram: phân bố điểm cho bài KT được chọn (Chart.js bar chart)
- Alert panel: danh sách HS yếu (đỏ, cần quan tâm) với link xem chi tiết
- Bảng kết quả: DataTable sortable (HS name, score, time, status)
- Export buttons: "Xuất PDF" + "Xuất Excel" → loading → download

 **Deliverable — Giai đoạn 7:** Dashboard analytics đầy đủ Student + Teacher. Biểu đồ, Knowledge Graph, HS yếu, xuất báo cáo PDF/Excel.



Giai đoạn 8: Parent Portal + Thông báo

- **Thời gian:** Ngày 51–57 (7 ngày)
- **Use Cases:** UC18, UC19, UC20, UC21, UC22
- **Phụ thuộc:** GĐ 7
- **Tóm tắt:** Phụ huynh liên kết con, giám sát kết quả, nhận thông báo in-app + email + web push.

1. Backend — Parent Service (BACKEND)

API Endpoints:

- **[POST]** /api/parent/link-student - Liên kết con bằng mã xác nhận
- **[GET]** /api/parent/children - Danh sách con đã liên kết

- **[GET]** /api/parent/children/:id/results - Kết quả bài KT của con
- **[GET]** /api/parent/children/:id/dashboard - Dashboard con (reuse student analytics)
- **[GET]** /api/parent/children/:id/analytics/strengths - Điểm mạnh/yếu con

Công việc chi tiết:

- Link student: Admin tạo mã (6 ký tự random) → parent nhập mã → verify → create parent_student record
- Reuse Student Analytics service nhưng query theo child student_id thay vì req.user.id

2. Backend — Notification Service (BACKEND)

API Endpoints:

- **[GET]** /api/notifications - Danh sách thông báo (pagination, filter is_read)
- **[PUT]** /api/notifications/:id/read - Đánh dấu đã đọc
- **[PUT]** /api/notifications/read-all - Đánh dấu tất cả đã đọc
- **[POST]** /api/push/subscribe - Đăng ký push notification (lưu subscription vào web_push_subscriptions)
- **[DELETE]** /api/push/unsubscribe - Hủy đăng ký push

Công việc chi tiết:

- Notification triggers (gọi từ các service khác):
 - → Khi HS submit bài → notify parent "Con bạn đã hoàn thành bài KT X, điểm: Y"
 - → Khi GV publish kết quả → notify students + parents
 - → Khi GV giao bài KT mới → notify students
- Email notifications: cài nodemailer + cấu hình Gmail SMTP / SendGrid
- Template email HTML cho: kết quả bài KT, xác nhận tài khoản, reset password
- Web Push: generate VAPID keys, implement web-push library
- Push flow: subscribe → save to DB → trigger event → web-push send → browser notification

3. Frontend — Parent Pages + Notifications (FRONTEND)

Công việc chi tiết:

- Tạo pages/parent/DashboardPage.tsx: chọn con (dropdown nếu >1) → reuse Student Dashboard components (readonly)
- Tạo pages/parent/LinkStudentPage.tsx: form nhập mã xác nhận 6 ký tự
- Tạo components/shared/NotificationBell.tsx: icon chuông ở topbar, badge đếm chưa đọc, click → dropdown list
- Notification dropdown: danh sách 10 gần nhất, click → navigate hoặc mark read, link "Xem tất cả"
- Tạo pages/common/NotificationsPage.tsx: full list + filter
- Push notification: register Service Worker, request permission on parent first login
- Handle push event: show browser notification → click → open app

 **Deliverable — Giai đoạn 8: Parent portal: liên kết con, dashboard con, thông**

báo in-app + email + web push khi có kết quả mới.



Giai đoạn 9: AI Thích ứng

- **Thời gian:** Ngày 58–64 (7 ngày)
- **Use Cases:** UC16, UC17
- **Phụ thuộc:** GĐ 7
- **Tóm tắt:** AI (GPT-4) sinh câu hỏi thích ứng từ câu trả lời sai, phiên luyện tập cá nhân hóa.

1. Backend — AI Service (BACKEND)

API Endpoints:

- **[POST]** /api/student/ai-practice/start - Tạo phiên AI từ bài KT đã làm (phân tích câu sai → gọi GPT-4)
- **[GET]** /api/student/ai-practice/:sessionId - Lấy câu hỏi AI đã sinh
- **[POST]** /api/student/ai-practice/:sessionId/answer - Trả lời câu hỏi AI
- **[GET]** /api/student/ai-practice/:sessionId/result - Kết quả phiên luyện tập
- **[GET]** /api/student/ai-practice - Lịch sử phiên AI

Công việc chi tiết:

- Cài OpenAI SDK: npm i openai
- Phân tích câu sai: lấy attempt_answers WHERE is_correct=false → group by topic → chọn topic yếu nhất
- Prompt engineering: system prompt = "Bạn là giáo viên THPT VN chuyên [subject]", user prompt = câu hỏi gốc + đáp án sai + yêu cầu tạo 5 câu mới cùng chủ đề
- Response parsing: yêu cầu GPT-4 trả JSON strict format → parse → validate → lưu ai_generated_questions
- Fallback: nếu OpenAI API timeout/error → chọn câu từ ngân hàng (cùng topic + difficulty ± 1)
- Rate limiting: max 10 phiên AI / student / ngày (config trong system_configs)
- Bull queue: queue AI requests nếu nhiều student đồng thời → process sequentially
- Cost tracking: log tokens used per request → admin monitoring

2. Frontend — AI Practice (FRONTEND)

Công việc chi tiết:

- Tạo pages/student/AIPracticePage.tsx: chọn bài KT đã làm → xem câu sai → nút "Luyện tập AI"
- Loading state: skeleton + animation "AI đang phân tích và tạo câu hỏi..."
- Interface làm bài AI: tương tự TakeExamPage nhưng đơn giản hơn (không timer, navigation giữa câu)
- Kết quả phiên AI: score, so sánh topic mastery trước/sau, gợi ý tiếp tục luyện tập
- Danh sách phiên AI đã thực hiện: date, topic, score, # questions



Deliverable — Giai đoạn 9: AI sinh câu hỏi thích ứng từ GPT-4 dựa trên câu sai.

Phiên luyện tập cá nhân hóa hoàn chỉnh.

Giai đoạn 10: Real-time + Polish

- **Thời gian:** Ngày 65–72 (8 ngày)
- **Use Cases:** Không có
- **Phụ thuộc:** GĐ 8, GĐ 9
- **Tóm tắt:** Socket.IO real-time, performance optimization, responsive design, dark mode, loading states, error handling UI.

1. Backend — Socket.IO Setup (BACKEND)

Công việc chi tiết:

- Cài socket.io + @socket.io/redis-adapter
- Tích hợp Socket.IO server vào Express app
- Redis adapter cho multi-instance support
- Auth middleware cho socket: verify JWT on connection
- Rooms strategy: user:{userId}, class:{classId}, exam:{examId}
- Events emit:
 - → exam:started — khi exam PUBLISHED (broadcast to assigned classes)
 - → exam:student-submitted — GV thấy real-time ai đã nộp
 - → notification:new — push notification in-app
 - → exam:time-warning — cảnh báo còn 5 phút
 - → dashboard:update — invalidate cache khi có data mới

2. Frontend — Socket.IO Client (FRONTEND)

Công việc chi tiết:

- Tạo hooks/useSocket.ts: connect on login, disconnect on logout, auto-reconnect
- Listen events: show toast/notification khi nhận event
- Exam page: real-time cập nhật "X/Y học sinh đã nộp" cho GV
- Notification bell: badge tự động tăng khi nhận notification:new

3. Performance Optimization (FULL-STACK)

Công việc chi tiết:

- Backend: Redis cache dashboard queries (TTL 5 phút), invalidate on submit
- Backend: Prisma query optimization — select only needed fields, avoid N+1 (use include wisely)
- Backend: Pagination everywhere (cursor-based cho large datasets)
- Backend: Gzip compression via compression middleware
- Frontend: React.lazy + Suspense cho mỗi page (code splitting)
- Frontend: React Query cacheTime + staleTime tuning
- Frontend: useMemo/useCallback cho heavy chart components

- Frontend: Image lazy loading + avatar size optimization
- Frontend: Bundle analysis (vite-plugin-visualizer) → loại bỏ unused deps

4. UI Polish (FRONTEND)

Công việc chi tiết:

- Responsive design: test tablet (768px) + mobile (375px) cho mọi page
- Dark mode: CSS variables + toggle button, lưu preference localStorage
- Loading skeletons: tạo skeleton components cho mỗi page type (table, card, chart)
- Toast notifications: react-hot-toast cho success/error/warning
- Empty states: illustration + message cho mọi danh sách rỗng
- Error boundaries: React ErrorBoundary wrapper cho mỗi route
- Page transitions: framer-motion fade/slide animations
- 404 page (Not Found), 403 page (Forbidden), 500 page (Server Error)
- Breadcrumbs navigation
- Keyboard shortcuts: Ctrl+K search, Esc close modal

 **Deliverable** — **Giai đoạn 10:** Real-time updates qua Socket.IO. Performance tối ưu. UI mượt mà, responsive, dark mode, loading states.



Giai đoạn 11: Testing + CI/CD

- **Thời gian:** Ngày 73–80 (8 ngày)
- **Use Cases:** Không có
- **Phụ thuộc:** GD 10
- **Tóm tắt:** Unit tests, integration tests, E2E tests. GitHub Actions CI/CD pipeline. Branch protection.

1. Backend — Unit Tests (Jest) (BACKEND)

Công việc chi tiết:

- Cài jest, ts-jest, @types/jest, supertest, @types/supertest
- Cấu hình jest.config.ts: TypeScript transform, test database
- Test Auth Service: login success/fail, token generation, password hash verify, refresh token, reset password flow
- Test Exam Service: create exam, add questions, publish, schedule logic
- Test Question Service: CRUD, import validation, filter logic
- Test Analytics Service: score calculation accuracy, aggregation correctness
- Test AI Service: prompt construction, response parsing, fallback logic
- Test middlewares: auth (valid/invalid/expired token), roleAuth, rateLimiter, validation
- Target: 70%+ code coverage cho business logic (services + controllers)

2. Backend — Integration Tests (Supertest) (BACKEND)

Công việc chi tiết:

- Setup test database (separate PostgreSQL DB hoặc Docker container)
- Tạo test helpers: createTestUser(), loginAsRole(), seedTestData()
- Test full flow: login → create question → create exam → assign → student start → submit → view result
- Test RBAC: student cannot POST /api/admin/users (403), teacher cannot DELETE user (403)
- Test validation: invalid email format (400), missing required fields (400)
- Test edge cases: submit after deadline, duplicate exam attempt, concurrent submissions
- Cleanup: truncate test data after each test suite

3. Frontend — Component Tests (Vitest) (FRONTEND)

Công việc chi tiết:

- Cài vitest, @testing-library/react, @testing-library/jest-dom, msw (Mock Service Worker)
- Test LoginPage: render form, validation errors, successful login redirect
- Test TakeExamPage: timer countdown, answer selection, auto-save trigger, submit confirmation
- Test hooks: useAuth (login/logout state changes), useExam (data fetching)
- Test shared components: QuestionCard, DifficultyBadge, NotificationBell
- MSW: mock API responses cho consistent testing

4. CI/CD Pipeline (GitHub Actions) (DEVOPS)

Công việc chi tiết:

- Tạo .github/workflows/ci.yml:
 - → Trigger: on push/PR to main, develop
 - → Job 1: Lint (ESLint) + Format check (Prettier) + TypeScript type check
 - → Job 2: Backend tests (Jest + Supertest) with PostgreSQL + Redis services
 - → Job 3: Frontend tests (Vitest)
 - → Job 4: Build check (TypeScript compile + Vite build)
 - → Job 5 (optional): Docker build check
 - Branch protection rules:
 - → main: require PR, require all checks pass, require 1 review
 - → develop: require checks pass
 - Cấu hình GitHub secrets: DB credentials, API keys (cho test environment)
-  **Deliverable — Giai đoạn 11:** Test suite: 70%+ coverage backend, component tests frontend. CI pipeline auto-run on push. Branch protection enabled.



Giai đoạn 12: Docker + Deploy + Monitoring

- **Thời gian:** Ngày 81–87 (7 ngày)
- **Use Cases:** Không có
- **Phụ thuộc:** GĐ 11
- **Tóm tắt:** Production Docker images, Nginx reverse proxy, SSL, Prometheus monitoring,

Grafana dashboards, Loki logging.

1. Dockerfiles Production (DEVOPS)

Công việc chi tiết:

- Tạo docker/Dockerfile.server: multi-stage build
 - Stage 1 (builder): npm ci → prisma generate → tsc build
 - Stage 2 (runner): copy dist + node_modules + prisma → CMD node dist/server.js
- Tạo docker/Dockerfile.client: multi-stage build
 - Stage 1: npm ci → vite build
 - Stage 2: nginx:alpine → copy dist → copy nginx.conf
- Tạo docker/nginx.conf: reverse proxy
 - / → client (static)
 - /api → server:3000
 - /socket.io → server:3000 (WebSocket upgrade)
- Tạo docker-compose.prod.yml với tất cả services
- Test: docker compose -f docker-compose.prod.yml up --build → toàn bộ stack chạy

Code Example:

docker-compose.prod.yml — 10 services

services:

```
nginx:      # Reverse proxy + SSL termination
server:     # Node.js Express app
client:     # Vite build served by Nginx
postgres:   # PostgreSQL 16 + volume + backup cron
pgbouncer:  # Connection pooling (max 100 connections)
redis:      # Cache + sessions + queue
minio:      # Object storage
prometheus: # Metrics scraping (15s interval)
grafana:    # Dashboards + alerting
loki:       # Log aggregation
promtail:   # Log shipping to Loki
```

2. Monitoring (Prometheus + Grafana) (DEVOPS)

Công việc chi tiết:

- Cài prom-client vào Express: default metrics + custom metrics
- Custom metrics: http_request_duration_seconds, active_users_total, exam_submissions_total, ai_requests_total
- Expose /metrics endpoint (chỉ internal access)
- Prometheus config: scrape targets (server, postgres-exporter, redis-exporter, node-exporter)

- Grafana dashboards:
 - → System Overview: CPU, RAM, disk, network
 - → Application: request rate, response time (p50/p95/p99), error rate
 - → Business: active users, exams created, submissions/hour
 - → Database: active connections, query duration, cache hit rate
- Alert rules: response_time_p95 > 2s, error_rate > 5%, disk_usage > 80%
- Alert channels: email + Slack webhook

3. Logging (Loki + Promtail) (DEVOPS)

Công việc chi tiết:

- Winston logger format: JSON structured logging {timestamp, level, message, requestId, userId, ...}
- Correlation ID middleware: mỗi request nhận unique requestId → gắn vào mọi log entry
- Log levels: error (bugs), warn (recoverable), info (business events), debug (dev only)
- Promtail config: collect từ Docker container logs → ship to Loki
- Grafana Log Explorer: filter by level, service, requestId, userId

4. Production Checklist (DEVOPS)

Công việc chi tiết:

- ☐ SSL certificate (Let's Encrypt / Certbot auto-renew)
- ☐ Environment variables: tất cả secrets qua .env (không hardcode)
- ☐ Database backup cron: pg_dump daily → gzip → MinIO "backups" bucket → retain 30 ngày
- ☐ Nginx: rate limiting (100 req/min general, 10 req/min login), gzip on, security headers
- ☐ CORS: whitelist production domain only
- ☐ Helmet: security headers (CSP, HSTS, X-Frame-Options)
- ☐ Redis: password required, maxmemory policy allkeys-lru
- ☐ PostgreSQL: max_connections qua PgBouncer, log slow queries > 1s
- ☐ MinIO: access policies per bucket (uploads: authenticated, reports: signed URLs)
- ☐ Health check endpoints: /health (server), /health/db, /health/redis
- ☐ Graceful shutdown: handle SIGTERM, close connections, finish in-flight requests
- ☐ Docker resource limits: memory + CPU per container

 **Deliverable — Giai đoạn 12:** Production deployment: Docker stack, SSL, monitoring dashboards, alerting, centralized logging, automated backup. PROJECT COMPLETE! 🎉